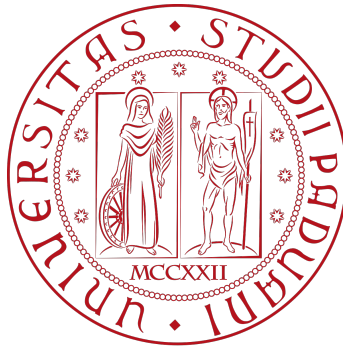




UNIVERSITÀ DEGLI STUDI DI PADOVA

Scuola di Scienze  
Corso di Laurea in Informatica

Progetto - Basi di Dati

# Banca del sangue

*Studenti:*

**Giovanni Veronese  
Sebastiano Lorenzi**

*Docente:*

**Prof. Massimiliano de Leoni**

**Anno Accademico: 2025-2026**

# Indice

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| <b>1</b> | <b>Abstract</b>                               | <b>1</b>  |
| <b>2</b> | <b>Analisi dei requisiti</b>                  | <b>1</b>  |
| 2.1      | Schema E-R . . . . .                          | 3         |
| <b>3</b> | <b>Progettazione concettuale</b>              | <b>4</b>  |
| <b>4</b> | <b>Progettazione logica</b>                   | <b>5</b>  |
| 4.1      | Eliminazione delle generalizzazioni . . . . . | 5         |
| 4.2      | Analisi delle ridondanze . . . . .            | 5         |
| 4.2.1    | Con ridondanza . . . . .                      | 6         |
| 4.2.2    | Senza ridondanza . . . . .                    | 6         |
| 4.3      | Scelta degli identificatori . . . . .         | 7         |
| 4.3.1    | Schema E-R ristrutturato . . . . .            | 7         |
| 4.4      | Schema relazionale . . . . .                  | 7         |
| <b>5</b> | <b>Implementazione PostgreSQL e query</b>     | <b>9</b>  |
| 5.1      | Definizione delle query . . . . .             | 9         |
| 5.1.1    | Query 1 . . . . .                             | 9         |
| 5.1.2    | Query 2 . . . . .                             | 10        |
| 5.1.3    | Query 3 . . . . .                             | 10        |
| 5.1.4    | Query 4 . . . . .                             | 11        |
| 5.1.5    | Query 5 . . . . .                             | 12        |
| 5.2      | Definizione degli indici . . . . .            | 12        |
| <b>6</b> | <b>Codice C</b>                               | <b>12</b> |
| 6.1      | Connessione al Database . . . . .             | 12        |
| 6.2      | Esecuzione delle Query . . . . .              | 13        |
| 6.3      | Elaborazione e Stampa dei Risultati . . . . . | 13        |

## 1 Abstract

In un sistema sanitario che è costantemente alla ricerca di nuovo sangue e plasma per le trasfusioni, una base di dati che garantisce una archiviazione e gestione corretta delle informazioni è essenziale, senza tralasciare sicurezza ed efficienza.

Il progetto ha la finalità di gestire un database sulle donazioni di sangue, dal momento in cui vengono create le singole sacche quindi dal **DONATORE**, al momento in cui vengono utilizzate per un **PAZIENTE**.

Il sistema distingue i centri prelievi dagli ospedali, permettendo di registrare i prelievi effettuati dai donatori negli appositi centri di raccolta, la creazione delle sacche e la rispettiva locazione, il loro stato di disponibilità e il loro eventuale trasferimento tra ospedali. Ogni sacca può essere utilizzata in una trasfusione, collegata a una specifica richiesta di sangue e a un determinato paziente.

## 2 Analisi dei requisiti

Questa sezione riassume i requisiti a cui deve sottostare la base di dati.

Per poter donare il sangue bisogna iscriversi al database **DONATORE**, fornendo le seguenti informazioni:

- Codice fiscale (**cf**) univoco che lo identifichi
- Un nominativo **nome** e **cognome**
- Viene assegnato un **gruppo sanguigno** [Valori possibili: 'A', 'B', 'AB', 'O']
- L'informazione del **fattore RH** specifica il gruppo sanguigno [Valori possibili: '+', '-']
- Il **numero di telefono** per contattare il centro o essere contattato

Un donatore uomo deve aspettare almeno 90 giorni tra una donazione e l'altra, per permettere al sangue di rigenerarsi e l'organismo di riprendersi. Una donatrice deve invece aspettare 180 giorni. Si può diventare donatori dai 18 ai 60 anni, mentre si può donare fino al compimento dei 65 anni, indipendentemente dal sesso. Le informazioni sul sesso o sulla data di nascita non sono necessarie nell'entità **DONATORE** in quanto sono intrinseche e calcolabili dal codice fiscale. Il donatore deve inoltre dimostrare di godere di buona salute e non aver soggiornato all'estero negli ultimi 6 mesi, in aggiunta di altri criteri che possono cambiare di giorno in giorno.

Un donatore può effettuare più prelievi in momenti differenti, può donare sangue intero o plasma a piacimento e in un centro di raccolta non deciso a priori.

È possibile effettuare una sola donazione alla volta e di conseguenza generare un **PRELIEVO** identificato da:

- Un **id** univoco e sequenziale
- Una **data** e un orario (precisione al secondo)

Un prelievo è di 450g di sangue intero oppure di 700g (al massimo) di plasma. La data precisa è necessaria per il calcolo della durata di vita utile della sacca. In questo progetto è stato stabilito il periodo di vita utile arbitrariamente conservativo di 365 giorni per il plasma, di 40 giorni per il sangue intero.

Ogni prelievo genera una sola sacca, e viene eseguito in un centro prelievi che rifornisce uno e un solo ospedale. Il **CENTRO\_PRELIEVI** ha i seguenti attributi:

- Un **nome** che può ripetersi solo se riferisce a un diverso ospedale

- Un indirizzo in cui è ubicato. Contiene: **via, numero civico, CAP, città, provincia**

Per mantenere una dimensione accademica è stato stabilito che le sacche prodotte vengano immediatamente trasferite al magazzino dell'unico ospedale di riferimento, con un tempo di trasporto virtualmente immediato. Il centro prelievi può eseguire più prelievi, e inviare all'ospedale di riferimento la **SACCA** prodotta, il centro del progetto, che è così identificata:

- Ha un univoco **id** incrementale come da norma europea
- Un **contenuto** [Valori possibili: 'Sangue intero', 'Plasma']. Ai fini del progetto, il sangue intero non viene diviso nelle sue emocomponenti come nella realtà.
- Un **gruppo sanguigno** [Valori possibili: 'A', 'B', 'AB', 'O']
- Un **fattore RH** che accompagna il gruppo sanguigno [Valori possibili: '+', '-']
- Una **data di scadenza** che in questo progetto è 365 giorni per il Plasma, 40 giorni per il Sangue intero
- Uno **stato della sacca** per tenere traccia della disponibilità [Valori possibili: 'Disponibile', 'In uso', 'Scartata', 'In transito', 'Esaurita']

Per mantenere il progetto adatto a una dimensione accademica è stato stabilito di ridurre la destinazione d'uso da più opzioni a una sola: la trasfusione in ambito ospedaliero, consapevoli del fatto che nella realtà la maggior parte del plasma viene utilizzato in ambito farmaceutico. Il sangue intero dopo un reale prelievo viene invece suddiviso nelle sue emocomponenti (globuli rossi, globuli bianchi, piastrine...), mentre nel progetto plasma e sangue vengono mantenuti egualmente inalterati fino alla destinazione d'uso finale.

Una volta prodotta, una sacca può andare incontro a diversi destini, tracciati dallo stato: può essere immagazzinata in un ospedale (Disponibile), essere prenotata per una trasfusione (In uso), scadere o venire arbitrariamente tolta dalle scorte per motivi medici (Scartata), essere oggetto a trasferimento in un altro ospedale (In transito), o essere utilizzata per una trasfusione (Esaurita). Di default, viene immediatamente conservata con stato 'Disponibile' nell'ospedale a cui il centro prelievi fa riferimento.

Ogni **OSPEDALE** ha i seguenti attributi:

- Un **id** univoco
- Un **nome** che non ha scopi logici
- Un indirizzo per i trasferimenti. Contiene: **via, numero civico, CAP, città, provincia**

Ogni ospedale ha un proprio deposito per immagazzinare le sacche sovradimensionato rispetto al numero di sacche (non necessario controllare il numero massimo di stoccaggio). Può ricevere o spedire le sacche, ricoverare pazienti e richiedere il sangue. Ogni **RICHIESTA\_SANGUE** è identificata da:

- Un **id richiesta** univoco e incrementale
- Un **codice fiscale del medico** che effettua la richiesta, in modo da avere sempre una persona di riferimento
- Un **codice fiscale del paziente** per il quale viene effettuato l'ordine
- Un **id ospedale di ricovero** dove verrà effettuata la trasfusione, che corrisponderà alla destinazione dell'eventuale trasferimento
- Una **data** in cui avviene la richiesta, precisione al secondo

- Una **quantità di sacche** richiesta, si assume che in unico ordine se le sacche da ordinare sono più di una abbiano lo stesso contenuto, gruppo sanguigno e fattore RH
- Si specifica l'**emocomponente** [Valori possibili: 'Sangue intero', 'Plasma']
- Un **reparto** di destinazione per sapere dove verrà utilizzato il sangue [Valori possibili: 'Pronto soccorso', 'Chirurgia', 'Oncologia ed ematologia', 'Ostetricia', 'Terapia intensiva e rianimazione', 'Degenza ordinaria']
- Uno **stato** della richiesta [Valori possibili: 'Non evasa', 'Evasa parzialmente', 'Soddisfatta', 'Annullata'] Default: 'Non evasa'

Il contenuto della sacca viene assegnato in base alle caratteristiche del paziente. La specificazione del reparto serve a tenere ordinate e separate le sacche con stato 'In uso' nel magazzino una volta arrivate. Lo stato si aggiorna in base alla disponibilità in magazzino: quando la richiesta è appena stata inserita viene inserito uno stato di default (Non evasa); se un ordine di più sacche alcune sono già state messe da parte e alcune devono ancora arrivare (Evasa parzialmente); se le sacche sono pronte in magazzino (Soddisfatta) o se si cambia idea e si rendono nuovamente disponibili le sacche (Annullata).

Ogni richiesta che non ha una giacenza in magazzino porta a un **TRASFERIMENTO**, con le seguenti informazioni:

- Un **id spedizione** univoca e incrementale
- Una **data di spedizione** in cui lascia fisicamente l'ospedale mittente
- Una **data di arrivo** in cui arriva all'ospedale destinatario

Ogni trasferimento contiene una sola sacca. Nel progetto c'è un limite tecnico scelto: non è possibile accorpare più sacche per una singola spedizione. Serve a raggiungere la quantità corretta delle sacche richieste che possono poi portare a una **TRASFUSIONE** che ha come attributi:

- **id trasfusione** univoco e incrementale
- Una **data** della trasfusione, precisione al secondo

Ogni trasfusione utilizza una sola sacca, e deriva da una richiesta di sangue dalla quale ricava il paziente a cui andrà effettuata la trasfusione. Ogni entità **PAZIENTE** deve fornire gli attributi prima di qualsiasi richiesta:

- Codice fiscale (**cf**) univoco che lo identifichi
- Un nominativo **nome e cognome**
- Viene assegnato un **gruppo sanguigno** [Valori possibili: 'A', 'B', 'AB', 'O']
- L'informazione del **fattore RH** specifica il gruppo sanguigno [Valori possibili: '+', '-']

Un paziente per essere accettato dal sistema non ha vincoli di età, a parte quello di essere in vita. È possibile che un paziente venga ricoverato in più ospedali nel corso del tempo (relazione molti a molti). Qui si conclude il ciclo di vita utile di una sacca.

## 2.1 Schema E-R

Figura 1

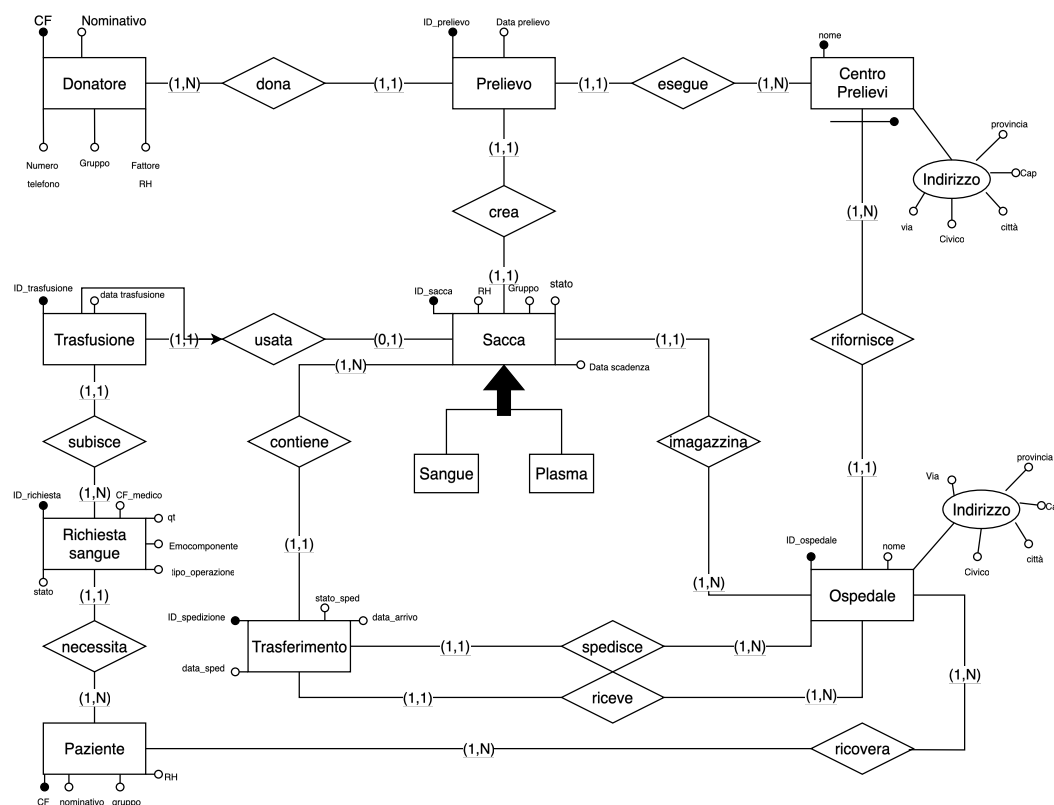


Figura 1: Diagramma Entità - Relazione del database per la gestione delle sacche di sangue

### 3 Progettazione concettuale

Dalla Figura 1, che mostra lo Schema E-R risultato dall'analisi dei requisiti, si ricavano le seguenti tabelle che elencano le entità e le relazioni nel database. Entità dello schema E-R:

- **DONATORE**(cf, nome, cognome, gruppo\_sanguigno, fattore\_rh, numero di telefono) rappresenta una persona che può donare
- **PAZIENTE**(cf, nome, cognome, gruppo\_sanguigno, fattore\_rh) rappresenta una persona che può ricevere una trasfusione
- **OSPEDALE**(id, nome, civico, via, città, provincia, CAP) rappresenta la struttura che immagazzina, invia, riceve sacche e ricovera pazienti
- **CENTRO\_PRELIEVI**(nome, civico, via, città, provincia, CAP) rappresenta il luogo, dipendente da un ospedale, dove vengono raccolti i prelievi
- **PRELIEVO**(id, data\_ora) rappresenta l'atto in cui viene estratto il sangue da un donatore
- **SACCA**(id, contenuto, gruppo\_sanguigno, fattore\_rh, data\_scadenza, stato\_sacca) rappresenta fisicamente la sacca di sangue intero o plasma
- **RICHIESTA\_SANGUE**(id, cf\_medico\_richiedente, data\_richiesta, quantità\_sacche, emocomponente\_richiesto, reparto\_destinazione, stato) rappresenta una richiesta di sacche effettuata da un medico
- **TRASFERIMENTO**(id, data\_spedizione, data\_arrivo) rappresenta la movimentazione logistica di una sacca tra due ospedali

- **TRASFUSIONE**(id, data\_operazione) rappresenta l'intervento conclusivo con cui si utilizza la sacca e termina il ciclo di vita

Oltre alle entità, nello schema concettuale è presente una relazione con attributi propri:

- **RICOVERA**(data\_inizio, data\_fine) rappresenta il periodo di permanenza di un paziente all'interno di un ospedale

## 4 Progettazione logica

In questa sezione viene illustrato il processo di ristrutturazione dello schema concettuale, con l'obiettivo di ottenere uno schema facilmente traducibile nel modello relazionale. In particolare, vengono analizzate le generalizzazioni presenti nel diagramma E-R, vengono scelti gli identificatori delle entità e infine viene effettuata la traduzione verso il modello relazionale.

### 4.1 Eliminazione delle generalizzazioni

Nel diagramma E-R è presente una sola generalizzazione, ovvero entità **SACCA**, suddivisa nelle due entità figlie **SANGUE\_INTERO** e **PLASMA**.

Poiché la distinzione tra le due categorie riguarda esclusivamente il contenuto della sacca e non introduce ulteriori attributi o relazioni, si è scelto di eliminare la generalizzazione accorpando le entità figlie nell'entità padre.

L'eliminazione viene effettuata introducendo nell'entità **SACCA** l'attributo 'Contenuto', il cui dominio è costituito dai valori:

- Sangue Intero
- Plasma

Questa soluzione consente di ridurre il numero di entità presenti nello schema, semplificando quindi la traduzione nel modello relazionale senza perdita di informazioni. Inoltre, evita la creazione di tabelle aggiuntive e riduce la complessità delle interrogazioni che coinvolgono le sacche, mantenendo invariata l'efficacia del modello.

### 4.2 Analisi delle ridondanze

Dall'analisi dello schema E-R emerge una possibile ridondanza relativa agli attributi 'gruppo\_sanguigno' e 'fattore\_rh' presenti nell'entità **SACCA**.

Informazioni che possono essere prelevate attraverso il percorso:

**SACCA → PRELIEVO → DONATORE**

Poiché ogni sacca deriva da un prelievo effettuato da un donatore e il gruppo sanguigno e il fattore RH del donatore coincidono con quelli della sacca prodotta.

La presenza di tali attributi nell'entità **SACCA** introduce una ridondanza informativa da analizzare per valutare se conviene mantenerla oppure no.

Si considerano le seguenti operazioni:

- **Operazione 1** (100 volte al giorno): registrazione di una nuova sacca in seguito di un prelievo.
- **Operazione 2** (300 volte al giorno): ricerca di sacche compatibili per una richiesta di sangue.

Si assumono i seguenti volumi:

| Concetto | Costrutto | Volume |
|----------|-----------|--------|
| DONATORE | E         | 300    |
| PRELIEVO | E         | 1000   |
| SACCA    | E         | 1000   |

#### 4.2.1 Con ridondanza

**Operazione 1** Costo =  $1 + 1 + 2 = 4$

| Concetto | Accessi | Tipo |
|----------|---------|------|
| PRELIEVO | 1       | L    |
| DONATORE | 1       | L    |
| SACCA    | 1       | S    |

**Operazione 2** Costo =  $N$  = volume concetto sacca = 1000

| Concetto | Accessi | Tipo |
|----------|---------|------|
| SACCA    | N       | L    |

**Costo totale** =  $100 \times 4 + 300 \times 1000 = 300.400$

#### 4.2.2 Senza ridondanza

**Operazione 1** Costo =  $1 + 1 + 2 = 4$

| Concetto | Accessi | Tipo |
|----------|---------|------|
| PRELIEVO | 1       | L    |
| DONATORE | 1       | L    |
| SACCA    | 1       | S    |

**Operazione 2** Costo =  $3N = 3000$

| Concetto | Accessi | Tipo |
|----------|---------|------|
| PRELIEVO | N       | L    |
| DONATORE | N       | L    |
| SACCA    | N       | S    |

**Costo totale** =  $100 \times 4 + 300 \times 3000 = 900.400$

Dall'analisi effettuata emerge che il mantenimento degli attributi 'gruppo\_sanguigno' e 'fattore\_rh' all'interno dell'entità **SACCA** comporta una ridondanza informativa, in quanto tali dati possono essere ricavati attraverso il percorso **SACCA** → **PRELIEVO** → **DONATORE**.



Tuttavia, il confronto tra i costi delle operazioni mostra come la presenza della ridondanza riduca significativamente il numero di accessi necessari durante le interrogazioni più frequenti, in particolare quelle finalizzate alla ricerca di sacche compatibili per le richieste di sangue. Inoltre, la memorizzazione diretta di tali informazioni nella sacca consente di limitare l'accesso ai dati anagrafici del donatore, garantendo una maggiore separazione tra dati clinici e dati personali.

Per questi motivi si è scelto di mantenere gli attributi 'gruppo\_sanguigno' e 'fattore\_rh' nell'entità **SACCA**, privilegiando l'efficienza delle interrogazioni e la tutela della riservatezza dei dati rispetto alla completa eliminazione della ridondanza.

### 4.3 Scelta degli identificatori

In questa fase vengono analizzati gli identificatori delle entità presenti nel modello, verificando che ciascuna istanza possa essere identificata in maniera univoca e stabile nel tempo.

Per le entità **DONATORE** e **PAZIENTE** è stato scelto come identificatore il codice fiscale. Tale attributo rappresenta un identificatore naturale, univoco a livello nazionale e immutabile nel tempo, risultando quindi adatto all'identificazione delle persone coinvolte nel sistema.

Per le entità Prelievo, Sacca, Trasferimento, Richiesta di sangue e Trasfusione si è scelto di utilizzare identificatori artificiali numerici ('id\_prelievo', 'id\_sacca', 'id\_trasferimento', 'id\_richiesta', 'id\_trasfusione'). Questa soluzione consente di garantire l'univocità delle istanze, essenziale in ambito medico, evitando l'utilizzo di chiavi composte da numerosi attributi e semplificando la definizione delle chiavi esterne nelle relazioni.

L'entità Ospedale viene identificata mediante un identificatore numerico ('id\_ospedale'). Sebbene il nome dell'ospedale possa risultare significativo, esso non garantisce necessariamente unicità e stabilità nel tempo; per questo motivo è stato preferito un identificatore artificiale.

L'entità Centro Prelievi è identificata mediante la coppia ('id\_ospedale', 'nome'). Tale scelta deriva dal fatto che ogni centro prelievi è gestito da un solo ospedale ed il nome del centro risulta univoco solamente all'interno dell'ospedale di appartenenza.

Infine, per l'entità derivata dalla relazione molti-a-molti Ricovero, è stato adottato come identificatore la combinazione ('cf\_paziente', 'id\_ospedale', 'data\_ricovero'). La presenza della data di ricovero permette infatti di distinguere eventuali ricoveri multipli dello stesso paziente presso il medesimo ospedale in momenti differenti.

La scelta effettuata privilegia identificatori stabili, di dimensione ridotta e facilmente utilizzabili nella definizione delle relazioni tra le diverse entità del sistema.

#### 4.3.1 Schema E-R ristrutturato

Di seguito la spiegazione che ha portato alla Figura 2

### 4.4 Schema relazionale

Di seguito tutte le tabelle rappresentate nello schema logico a seguito della ristrutturazione E-R:

- **OSPEDALE** (id, nome, civico, via, città, provincia, cap)
- **CENTRO\_PRELIEVI** (nome, civico, via, città, provincia, cap, id\_ospedale)
  - CENTRO\_PRELIEVI.id\_ospedale → OSPEDALE.id
- **DONATORE** (cf, nome, cognome, gruppo\_sanguigno, fattore\_rh, numero\_di\_telefono)
- **PAZIENTE** (cf, nome, cognome, gruppo\_sanguigno, fattore\_rh)

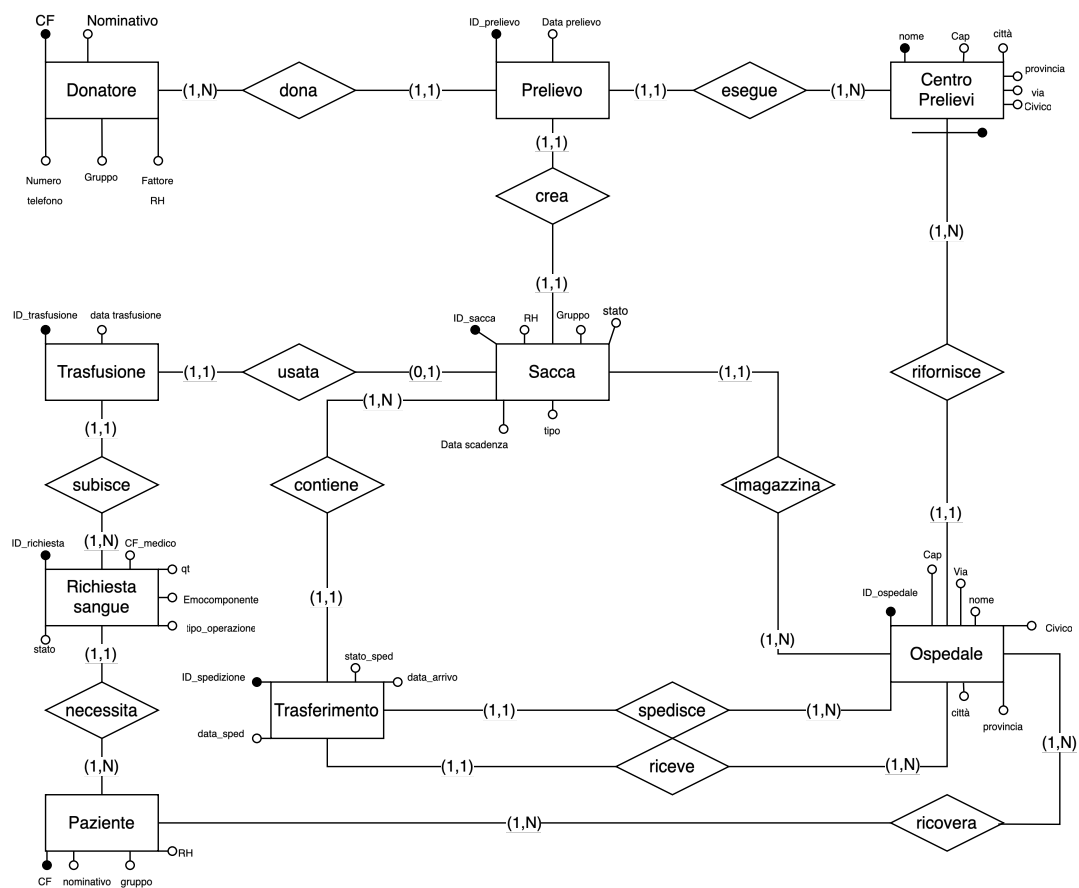


Figura 2: Diagramma Entità - Relazione ristrutturato

- **RICOVERA** (cf\_paziente, id\_ospedale, data\_inizio, data\_fine)
  - RICOVERA.cf\_paziente → PAZIENTE.cf
  - RICOVERA.id\_ospedale → OSPEDALE.id
- **PRELIEVO** (id, cf\_donatore, nome\_centro\_prelievi, id\_ospedale, data\_ora)
  - PRELIEVO.cf\_donatore → DONATORE.cf
  - PRELIEVO.(id\_ospedale, nome\_centro\_prelievi) → CENTRO\_PRELIEVI.(id\_ospedale, nome)
- **SACCA** (id, contenuto, gruppo\_sanguigno, fattore\_rh, id\_ospedale, id\_prelievo, data\_scadenza, stato\_sacca)
  - SACCA.id\_prelievo → PRELIEVO.id
  - SACCA.id\_ospedale → OSPEDALE.id
- **TRASFERIMENTO** (id, id\_sacca, id\_ospedale\_mittente, id\_ospedale\_destinatario, data\_spedizione, data\_arrivo)
  - TRASFERIMENTO.id\_sacca → SACCA.id
  - TRASFERIMENTO.id\_ospedale\_mittente → OSPEDALE.id
  - TRASFERIMENTO.id\_ospedale\_destinatario → OSPEDALE.id
- **RICHIESTA\_SANGUE** (id, cf\_medico\_richiedente, cf\_paziente, id\_ospedale\_ricovero, data\_richiesta, quantita\_sacche, emocomponente\_richiesto, reparto\_destinazione, stato)
  - RICHIESTA\_SANGUE.cf\_paziente → PAZIENTE.cf
  - RICHIESTA\_SANGUE.id\_ospedale\_ricovero → OSPEDALE.id
- **TRASFUSIONE** (id, data\_operazione, id\_sacca, id\_richiesta)
  - TRASFUSIONE.id\_sacca → SACCA.id
  - TRASFUSIONE.id\_richiesta → RICHIESTA\_SANGUE.id

## 5 Implementazione PostgreSQL e query

Il file banca\_del\_sangue.sql contiene il codice SQL usato per la generazione delle tabelle, il loro popolamento e il codice per la definizione delle query utilizzate

### 5.1 Definizione delle query

#### 5.1.1 Query 1

Controllare gli ospedali con il numero di sacche inferiore ad un limite imposto da utente

```

1  SELECT o.nome, s.gruppo_sanguigno, s.fattore_rh, COUNT(*) AS numero_sacche
2  FROM SACCA s JOIN OSPEDALE o ON s.id_ospedale = o.id
3  WHERE s.stato_sacca = 'Disponibile'
4  GROUP BY o.nome, s.gruppo_sanguigno, s.fattore_rh
5  HAVING COUNT(*) < $1
6  ORDER BY o.nome, s.gruppo_sanguigno, s.fattore_rh;
```

|    | nome<br>character varying (128) | gruppo_sanguigno<br>gruppo_sanguigno_enum | fattore_rh<br>fattore_rh_enum | numero_sacche<br>bigint |
|----|---------------------------------|-------------------------------------------|-------------------------------|-------------------------|
| 1  | Clinica Madonna della Salute    | O                                         | +                             | 1                       |
| 2  | Clinica Santa Rita              | A                                         | +                             | 1                       |
| 3  | Clinica Villa Berica            | A                                         | -                             | 1                       |
| 4  | Clinica Villa Berica            | O                                         | -                             | 1                       |
| 5  | Ospedale Alto Vicentino         | A                                         | -                             | 1                       |
| 6  | Ospedale Alto Vicentino         | B                                         | +                             | 2                       |
| 7  | Ospedale Borgo Trento           | A                                         | +                             | 1                       |
| 8  | Ospedale Borgo Trento           | O                                         | +                             | 2                       |
| 9  | Ospedale Ca' Foncello           | B                                         | +                             | 2                       |
| 10 | Ospedale Città di Jesolo        | A                                         | -                             | 1                       |

Figura 3: Risultato o esecuzione della Query 1, con parametro HAVING COUNT = 3

### 5.1.2 Query 2

Seleziona la quantità di sacche per provincia divise per gruppo

```

1  SELECT o.provincia, s.gruppo_sanguigno, fattore_rh, COUNT(*) AS num_scorte
2  FROM OSPEDALE o JOIN SACCA s ON o.id = s.id_ospedale
3  WHERE s.stato_sacca = 'Disponibile'
4  GROUP BY o.provincia, s.gruppo_sanguigno, s.fattore_rh
5  ORDER BY o.provincia, s.gruppo_sanguigno, s.fattore_rh;

```

|    | provincia<br>character (2) | gruppo_sanguigno<br>gruppo_sanguigno_enum | fattore_rh<br>fattore_rh_enum | num_scorte<br>bigint |
|----|----------------------------|-------------------------------------------|-------------------------------|----------------------|
| 1  | BL                         | A                                         | +                             | 11                   |
| 2  | BL                         | B                                         | +                             | 1                    |
| 3  | BL                         | O                                         | +                             | 9                    |
| 4  | BL                         | O                                         | -                             | 1                    |
| 5  | PD                         | A                                         | +                             | 12                   |
| 6  | PD                         | A                                         | -                             | 7                    |
| 7  | PD                         | B                                         | +                             | 8                    |
| 8  | PD                         | O                                         | +                             | 38                   |
| 9  | PD                         | O                                         | -                             | 7                    |
| 10 | RO                         | A                                         | +                             | 12                   |

Figura 4: Risultato o esecuzione della Query 2

### 5.1.3 Query 3

Trova l'ID, il nome, il reparto che ha utilizzato più sangue

```

1 DROP VIEW IF EXISTS num_sacche_reparto;
2 CREATE VIEW num_sacche_reparto AS
3 SELECT o.id, o.nome, rs.reparto_destinazione, SUM(rs.quantita_sacche) AS
   Num_sacche
4 FROM RICHIESTA_SANGUE rs JOIN OSPEDALE o ON rs.id_ospedale_ricovero = o.id
5 GROUP BY o.id, o.nome, rs.reparto_destinazione
6 ORDER BY Num_sacche DESC;
7 SELECT id, nome, reparto_destinazione, Num_sacche
8 FROM num_sacche_reparto
9 WHERE Num_sacche = (SELECT MAX(Num_sacche) FROM num_sacche_reparto);

```

|   | id<br>integer | nome<br>character varying (128) | reparto_destinazione<br>reparto_enum | num_sacche<br>bigint |
|---|---------------|---------------------------------|--------------------------------------|----------------------|
| 1 | 1584          | Ospedale Mater Salutis          | Ostetricia                           | 28                   |

Figura 5: Risultato o esecuzione della Query 3

#### 5.1.4 Query 4

Trova tutti i donatori di un determinato gruppo sanguigno che hanno donato di più

```

1 DROP VIEW IF EXISTS donazioni_per_donatore;
2 CREATE VIEW donazioni_per_donatore AS
3 SELECT d.cf, d.nome, d.cognome, d.gruppo_sanguigno, d.fattore_rh, COUNT(p.id
   ) AS numero_prelievi
4 FROM DONATORE d JOIN PRELIEVO p ON d.cf = p.cf_donatore
5 GROUP BY d.cf, d.nome, d.cognome, d.gruppo_sanguigno, d.fattore_rh;
6 SELECT *
7 FROM donazioni_per_donatore d1
8 WHERE numero_prelievi =(SELECT MAX(d2.numero_prelievi)
9 FROM donazioni_per_donatore d2
10 WHERE d2.gruppo_sanguigno = d1.gruppo_sanguigno
11 AND d2.fattore_rh = d1.fattore_rh)
12 ORDER BY gruppo_sanguigno, fattore_rh;

```

|    | cf<br>character (16) | nome<br>character varying (128) | cognome<br>character varying (128) | gruppo_sanguigno<br>gruppo_sanguigno_enum | fattore_rh<br>fattore_rh_enum | numero_prelievi<br>bigint |
|----|----------------------|---------------------------------|------------------------------------|-------------------------------------------|-------------------------------|---------------------------|
| 1  | FMBVNR79P49M31...    | Simona                          | Fioretti                           | A                                         | +                             | 4                         |
| 2  | TNFLRL79D23D205X     | Francesco                       | Carmen                             | A                                         | +                             | 4                         |
| 3  | TMRPBN05B46L21...    | Fabiola                         | Molina                             | A                                         | +                             | 4                         |
| 4  | VQNTNT93C01M31...    | Luca                            | Tufano                             | A                                         | +                             | 4                         |
| 5  | VRMNTT07L66M31...    | Francesca                       | Giovannetti                        | A                                         | +                             | 4                         |
| 6  | RPLNTL82C55M312S     | Francesca                       | Ricci                              | A                                         | +                             | 4                         |
| 7  | QNGVRE71T51D205V     | Francesca                       | Gualtieri                          | A                                         | +                             | 4                         |
| 8  | TLPMLT76L16M312X     | Vincenzo                        | Longo                              | A                                         | +                             | 4                         |
| 9  | MTGRLF01B71D205...   | Beatrice                        | Lombardi                           | A                                         | +                             | 4                         |
| 10 | GNGFCT77P09H50...    | Giacomo                         | Forleo                             | A                                         | +                             | 4                         |

Figura 6: Risultato o esecuzione della Query 4

### 5.1.5 Query 5

Trova il nome degli ospedali che ricevono più sacche di quante ne spediscono

```

1  SELECT o.id, o.nome, COUNT(t_in.id_sacca) AS sacche_ricevute, COUNT(t_out.
   id_sacca) AS sacche_spedite, COUNT(t_in.id_sacca) - COUNT(t_out.id_sacca) AS
   differenza
2  FROM ospedale o LEFT JOIN TRASFERIMENTO t_in ON o.id = t_in.
   id_ospedale_mittente
3  LEFT JOIN TRASFERIMENTO t_out ON o.id = t_out.id_ospedale_destinatario
4  GROUP BY o.id, o.nome
5  HAVING COUNT(t_in.id_sacca) > COUNT(t_out.id_sacca)
6  ORDER BY differenza DESC;

```

|   | id<br>[PK] integer | nome<br>character varying (128) | sacche_ricevute<br>bigint | sacche_spedite<br>bigint | differenza<br>bigint |
|---|--------------------|---------------------------------|---------------------------|--------------------------|----------------------|
| 1 | 258                | Ospedale Ca' Foncello           | 3                         | 0                        | 3                    |
| 2 | 369                | Clinica San Giovanni            | 1                         | 0                        | 1                    |

Figura 7: Risultato o esecuzione della Query 5

## 5.2 Definizione degli indici

È stato creato l'indice *idx\_prelievo\_cf\_donatore* sull'attributo 'cf\_donatore' della tabella **PRELIEVO**. Questo attributo è chiave esterna verso la tabella **DONATORE** e permette un significativo risparmio di calcolo evitando le frequenti operazioni di JOIN tra le due tabelle (come mostrato nella query 4).

Il trade off considerato è tra lo spazio aggiuntivo in memoria insieme all'overhead durante le operazioni di scrittura e la forte diminuzione dei tempi di ricerca, che giustifica l'utilizzo dell'indice in quanto il tempo è significativamente ridotto a discapito di un piccolo impiego di memoria extra.

## 6 Codice C

Il file `codice_banca_del_sangue.c` è strutturato in diverse parti, dalla connessione al database fino all'esecuzione delle query e alla visualizzazione finale dei dati.

### 6.1 Connessione al Database

I parametri di accesso (Host, Database, Utente, Password, Porta) vengono richiesti all'utente a runtime. Il programma si occupa di assemblare la stringa di connessione tramite la funzione `sprintf` e di avviare la connessione al DBMS invocando `PQconnectdb`. Lo stato della connessione viene immediatamente validato:

```

char conninfo[512];
sprintf(conninfo, "user=%s password=%s dbname=%s hostaddr=%s port=%d", PG_user, PG_pw, PG_db,
        PG_host, PG_port);

PGconn *conn; // Esegue la stringa di connessione
conn = PQconnectdb(conninfo);

if(PQstatus(conn) != CONNECTION_OK){

```

```

    printf("errore di connessione: %s\n", PQerrorMessage(conn));
    do_exit(conn);
}

```

## 6.2 Esecuzione delle Query

Un apposito menu ciclico consente all'utente di selezionare e mandare in esecuzione la query (inserendo un valore da 1 a 5, oppure 0 per terminare il programma). Al fine di ottimizzare le interrogazioni ed evitare errori, query dinamiche (come `query1`) vengono preparate in anticipo ed eseguite con passaggio di parametri:

```

// Prepara la query a Postgre
res = PQprepare(conn, "query1", query, 1, NULL);

// Inietta il parametro e la esegue
const char *paramValues[1] = { limite_str };
res = PQexecPrepared(conn, "query1", 1, paramValues, NULL, NULL, 0);

```

## 6.3 Elaborazione e Stampa dei Risultati

Ottenuta la risposta `PQresult`, la funzione ausiliaria `printResult` ne processa il contenuto stampandolo. Vengono utilizzate le funzioni dell'API `libpq` per estrarre le metainformazioni e le stringhe risultanti tramite `PQgetvalue`:

```

int tuple = PQntuples(res);
int campi = PQnfields(res);

// Stampa i valori di ogni riga e colonna
for(int i = 0; i < tuple; i++){
    for(int j = 0; j < campi; j++){
        printf("%-*s", COL_WIDTH, PQgetvalue(res, i, j));
    }
    printf("\n");
}

```

Al termine di ogni operazione sui dati, o in caso di anomalie riscontrate dalla funzione di controllo `checkResult`, la memoria allocata dalla libreria viene liberata invocando `PQclear(res)`. Alla scelta dell'utente di uscire dal programma, la connessione col DBMS viene chiusa in modo pulito invocando `PQfinish(conn)`.